

Mock Bank Demo — Brief

You build a fake bank app with a **customer side** (a customer makes transactions) and a **bank-admin side** (which automatically flags risky transactions, sends them to our system **Aegis**, and shows the reports we send back). We build everything in between.

What Aegis is (1 minute)

Banks/fintechs in India must file a **Suspicious Transaction Report (SAR)** with the regulator when they spot a suspicious transaction. **Aegis automates writing that report** — a bank sends us a flagged transaction, and Aegis returns a ready report that cites the bank's own AML policy.

For the demo we have no real bank, so **your app plays the bank**.

The flow (simple)

1. **A customer makes a transaction** (on the customer side of your app).
 2. **The bank's risk rules check it automatically**. If it's **risky**, the app **automatically sends it to Aegis** as JSON (with our address + an API key) — no human action.
 3. **Aegis does its thing** — masks personal data, finds the relevant policy, and an AI drafts the SAR. A **compliance officer** (the **bank's** officer, reviewing on the Aegis dashboard — we run this side for the demo) **reviews & approves it** (a human gate; may be automated later).
 4. **Aegis sends the finished report back** as a **goAML JSON + a PDF**.
 5. **The bank shows it** in its inbox.
-

What you build — two sides

1. Customer side (where transactions come from)

- A simple banking screen where a **customer makes a transaction** (amount, recipient, type → "Send"). This just creates the transaction.

2. Bank admin / compliance side (a monitoring dashboard — the admin only *watches*)

- **Risk rules (automatic)** — when a transaction comes in, the bank's hardcoded rules check it **automatically**. If it's **risky**, the app **automatically POSTs it to Aegis** — no human clicks anything. (This is how a real bank's monitoring system works.)
- **Monitoring view** — a list of the flagged transactions with their status (**sent** → **processing** → **report received**).
- **Inbox** — the reports Aegis sends back, displayed (goAML JSON + a button to open the PDF).
- (*Optional, for now*) The admin can take an action on a received report — e.g. mark it **Filed** or **Flag account**. Keep this simple; it's not required for the demo.

The admin **does not** send anything to Aegis by hand — the risky-transaction POST is **automatic**. And the report is **never sent to the flagged customer** (that's "tipping-off," which is illegal in AML). The

admin only *files* it or *flags the account* — never notifies the customer.

Base the risk rules on common red flags (pick any, combine a couple): amount just under ₹10,00,000 (structuring) · high-risk type (**INTERNATIONAL_WIRE**, **CRYPTO_PURCHASE**, **HAWALA**, **FOREX_TRANSFER**) · large refund/reversal · dormant account suddenly active · shady counterparty bank · round-number amount · many transactions in a short time.

You do **not** build any masking, AI, SAR drafting, or the officer approval — that's us. You build the **customer side** + the **bank-admin side** (rules, send, inbox).

Who the report is for (important context)

The report (SAR) is **not for the customer** — it's a **legal filing the bank submits to the regulator, FIU-India**, in the **goAML** format. So:

- The **customer NEVER sees it**. Telling a flagged customer is "tipping-off," which is **illegal** in AML. So there is **no "send to customer"** anywhere.
- The report is written **for the regulator** (it explains the suspicion to FIU-India). That's why we output **goAML JSON** — the exact format FIU accepts.
- On the **admin side**, the report arrives → the admin reviews it → the admin **files it with FIU** (for the demo, a mock **"File with FIU"** button). The admin may also flag/freeze the account internally — but **never notifies the customer**.

So the full chain is: **customer makes a transaction** → **bank flags it** → **Aegis drafts the SAR (for FIU)** → **admin reviews** → **admin files it with FIU-India**. The customer is never in this loop.

Constraints (important)

- **Everything is local. Nothing is deployed.** It all runs on one machine.
- **You deliver a ZIP** with a short README and run steps (`npm install && npm run dev` or similar). It must run offline, self-contained.
- **We run the Aegis backend ourselves** (we "paste it in" on our side) at `http://localhost:8000`. You only build the bank app.
- **Ports:** our Aegis frontend uses **5173**, so **your bank app must use a different port** (e.g. **5174** or **3000**). If you build a small receiver for the inbox, run it on another port (e.g. **8001**).
- **Make the URLs configurable** (env values), e.g. `AEGIS_BASE_URL=http://localhost:8000` and your `BANK_WEBHOOK_URL`, so we can wire the two sides together on our machine.
- Keep the stack simple (plain React/Vite or even HTML+JS is fine).

How to talk to Aegis

A) Send a transaction (your app → Aegis)

```
POST http://localhost:8000/api/v1/ingest/
Headers:
  X-API-Key:    <we give you this>
```

```
X-Tenant-ID: TEN-0001          (we give you this)
Content-Type: application/json
```

Body (example — a risky international wire just under the threshold):

```
{
  "customer": { "full_name": "Rohan Mehta", "id": "CUST-884213" },
  "account": { "number": "5012-7788-2231" },
  "txn": { "ref_id": "TXN-001", "amount": 945000, "currency": "INR",
    "type": "INTERNATIONAL_WIRE", "direction": "OUTBOUND",
    "timestamp": "2026-04-16T11:42:07+05:30" },
  "counterparty": { "account": "AE0703...", "name": "Gulf Horizon FZE", "bank":
    "Emirates NBD" },
  "metadata": { "ip": "203.0.113.57", "device_id": "dev-9f3c1a77" },
  "risk": { "score": 88, "reason": "Large outbound transfer just below the
    reporting threshold." }
}
```

Response:

```
{ "status": "success", "alert_id": "....", "risk_score": 100,
  "message": "Ingested successfully. SAR generation triggered." }
```

(**risk.score** is the score *your* rules produced. The **id**, names, account, etc. are just demo data you make up.)

B) Receive the report (Aegis → your app)

After the officer approves, Aegis POSTs the finished report to your receiver URL. The body is **JSON** containing the **goAML report data + a link to the PDF**:

```
{
  "event": "sar.approved",
  "sar_id": "SAR-....",
  "goaml_str": { "report": { "report_code": "STR", "...": "..." } },
  "pdf_url": "http://localhost:8000/files/sar/SAR-....pdf"
}
```

Your inbox shows the fields and a button to open **pdf_url**. (We'll give you a sample of this payload so you can build the inbox before we wire it live.)

Note: for the local demo you **don't** need to verify the webhook source. (In production that's done with a signature — not needed here.)

What we'll give you

- The **API key** + **tenant id** (TEN-0001) + the confirmed **base URL**.
 - A **sample report payload** so you can build the inbox early.
-

TL;DR

Build a **bank app with two sides**: a **customer side** (make a transaction) and a **bank-admin side** (risk rules flag it → send risky ones to Aegis → receive the report back as goAML JSON + PDF in an inbox). All local, delivered as a ZIP, on ports other than 5173. We do everything in between.